Optical_System_for_DPD_LEO

March 25, 2026

[1]:
```
import numpy as np
from scipy import signal
import tensorflow as tf
from tensorflow.keras import layers, Model, initializers, Sequential
from optic.models.devices import mzm, photodiode, edfa, iqm, coherentReceiver,␣
 ↪pdmCoherentReceiver, basicLaserModel
from optic.models.channels import linearFiberChannel, ssfm
from optic.comm.modulation import modulateGray, grayMapping
from optic.comm.sources import bitSource, symbolSource
from optic.dsp.core import upsample, pulseShape, pnorm, anorm, signalPower,␣
 ↪firFilter, decimate, symbolSync,phaseNoise

try:
    from optic.dsp.coreGPU import checkGPU
    if checkGPU():
        from optic.dsp.coreGPU import firFilter
    else:
        from optic.dsp.core import firFilter
except ImportError:
    from optic.dsp.core import firFilter

from optic.utils import parameters, dBm2W, ber2Qfactor
from optic.plot import eyediagram, pconst, plotPSD
import matplotlib.pyplot as plt
from scipy.special import erfc
from tqdm.notebook import tqdm
import scipy as sp
import scipy.constants as const

try:
    from optic.models.modelsGPU import manakovSSF
except:
    from optic.models.channels import manakovSSF

from optic.dsp.equalization import edc, mimoAdaptEqualizer, ffe
from optic.dsp.carrierRecovery import cpr
```

```
from optic.comm.metrics import fastBERcalc, monteCarloGMI, monteCarloMI,␣
 ↪calcEVM, bert
from optic.dsp.clockRecovery import gardnerClockRecovery


import logging as logg
logg.basicConfig(level=logg.INFO, format='%(message)s', force=True)
import time
```

[2]:
```
from IPython.core.display import HTML
from IPython.core.pylabtools import figsize

HTML("""
<style>
.output_png {
    display: table-cell;
    text-align: center;
    vertical-align: middle;
}
</style>
""")
```

[2]:
```
<IPython.core.display.HTML object>
```

[3]:
```
#␣
 ↪-----------------------------------------------------------------------------------------
# Power Amplifier (PA) Functions
#␣
 ↪-----------------------------------------------------------------------------------------

def rapp_pa(x, Vsat, p=2.0):
    """
    Memoryless Rapp PA model for complex baseband input.

    Parameters
    ----------
    x : np.ndarray
        Complex input waveform in volts.
    Vsat : float
        Saturation voltage/amplitude.
    p : float
        Rapp smoothness exponent.

    Returns
    -------
    y : np.ndarray
        Output after memoryless PA nonlinearity.
```

```
    """
    mag = np.abs(x)
    gain = 1.0 / (1.0 + (mag / Vsat) ** (2 * p)) ** (1.0 / (2 * p))
    return x * gain


def butter_lpf_complex(x, Fs, f3dB, order=3):
    """
    Apply an order-N Butterworth LPF to a complex baseband waveform.
    """
    wn = f3dB / (Fs / 2)

    if wn >= 1.0:
        raise ValueError(
            f"Butterworth cutoff must be below Nyquist. Got f3dB={f3dB/1e9:.2f}␣
 ↪GHz, "
            f"Fs/2={(Fs/2)/1e9:.2f} GHz."
        )

    b, a = signal.butter(order, wn, btype='low')

    y_i = signal.lfilter(b, a, np.real(x))
    y_q = signal.lfilter(b, a, np.imag(x))

    return y_i + 1j * y_q


def pa_model_physical(x, Fs, gain_linear, BLPF_enable, BO_dB=5.0, p=2.0,␣
 ↪f3dB=10e9, order=3):
    """
    Physical PA model:
        x -> linear gain -> Rapp compression -> Butterworth LPF

    Parameters
    ----------
    x : np.ndarray
        Complex baseband input waveform (dimensionless DSP waveform).
    Fs : float
        Sample rate [Hz].
    gain_linear : float
        Small-signal linear voltage gain. This sets the nominal IQM drive level.
    BO_dB : float
        Back-off in dB, used to set Vsat relative to the RMS value AFTER gain.
    p : float
        Rapp exponent.
    f3dB : float
        PA 3-dB bandwidth [Hz].
```

```
    order : int
        Butterworth filter order.

    Returns
    -------
    y : np.ndarray
        PA output waveform in volts, ready to drive the IQM.
    info : dict
        Diagnostic information.
    """

    # 1) Linear gain stage -> now waveform is in volts
    x_amp = gain_linear * x

    # 2) Compute RMS after gain
    Vrms_in = np.sqrt(np.mean(np.abs(x_amp) ** 2))

    # 3) Saturation voltage from back-off
    Vsat = Vrms_in * 10 ** (BO_dB / 20)

    # 4) Nonlinear compression
    y_nl = rapp_pa(x_amp, Vsat=Vsat, p=p)

    # 5) PA bandwidth limitation
    if BLPF_enable:
        y = butter_lpf_complex(y_nl, Fs=Fs, f3dB=f3dB, order=order)
    else:
        y = y_nl

    info = {
        "gain_linear": gain_linear,
        "Vrms_in_V": Vrms_in,
        "Vsat_V": Vsat,
        "BO_dB": BO_dB,
        "p": p,
        "f3dB_Hz": f3dB,
        "peak_out_V": np.max(np.abs(y)),
        "rms_out_V": np.sqrt(np.mean(np.abs(y) ** 2)),
    }

    return y, info
```

[4]:
```
#␣
 ↪-----------------------------------------------------------------------------------------
# Parameters Configuration of Optical System
#␣
 ↪-----------------------------------------------------------------------------------------
```

```
# ------------------------------------------------
# Transmitter Parameters
#-------------------------------------------------

# 1) General Parameters
SpS = 16                  # samples per symbol
Rs = 32e9                 # symbol rate [baud]
Fs = Rs * SpS             # sampling frequency [Hz]
M = 16                    # QPSK -> M=4, constType='qam'
nBits = 400000            # total bits to generate
rollOff = 0.01            # RRC roll-off
nFilterTaps = 1024        # RRC filter taps
mzmScale = 0.7            # IQM drive scale = "modulation depth" (0.5 is default␣
 ↪in OptiCommPy)
Vpi = 2                   # IQM pi Voltage  (2V is defualt in OpticommPy)
P_launch_dBm = 0          # desired launched optical power [dBm] -- for 4QAM try␣
 ↪6 dbm -- for 16QAM 0 was good
laserLinewidth = 100e3    # Hz (set e.g. 100e3 for phase noise), setting␣
 ↪laserLineWidth to 0, models the ideal case

# 2) Symbol Source Parameters
paramSymb = parameters()
paramSymb.nSymbols = int(nBits // np.log2(M))  # symbols = bits / log2(M)
paramSymb.M = M
paramSymb.constType = "qam"                    # 'qam' with M=4 -> QPSK
paramSymb.dist = "uniform"                     # uniform symbol probabilities
paramSymb.seed = 444
paramSymb.shapingFactor = 0

constSymb = grayMapping(paramSymb.M, paramSymb.constType)
if paramSymb.dist == "uniform":
   px = np.ones(paramSymb.M) / paramSymb.M
elif paramSymb.probDist == "maxwell-boltzmann":
   px = np.exp(-paramSymb.shapingFactor * np.abs(constSymb) ** 2)
   px = px / np.sum(px)
else:
   raise ValueError("Invalid probability distribution.")
paramSymb.px = px

# 3) UpSampling and FIR Parameters
paramPulse = parameters()
paramPulse.pulseType = "rrc"
paramPulse.nFilterTaps = nFilterTaps
paramPulse.rollOff = rollOff
paramPulse.SpS = SpS
```

```
# 4) IQM Parameters
paramIQM = parameters()
paramIQM.Vpi = Vpi
paramIQM.VbI = -Vpi
paramIQM.VbQ = -Vpi
paramIQM.Vphi = Vpi/2

# 5) Optical Carrier / LO field (Ein)
sigTx_length = paramSymb.nSymbols * SpS
if laserLinewidth and laserLinewidth > 0:
    phi_pn = phaseNoise(laserLinewidth, sigTx_length, 1 / Fs, seed=123)
    sigLO = np.exp(1j * phi_pn)
else:
    sigLO = np.ones_like(sigTx_length, dtype=complex)

# 6) PA parameters
PA_enable = True     # enable flag. If "False", the PA is skipped
PA_BO_dB = 3         # 7 = weak, 5 = moderate, 3 = strong
PA_p = 2.0           # paper says typically p=2
PA_f3dB = 18.5e9     # frequency of 3 dB
BLPF_enable = True


# ------------------------------------------------
# Channel Parameters
#-------------------------------------------------

# 1) Optical Channel Parameters
paramCh = parameters()
paramCh.Ltotal = 80       # total link distance [km]
paramCh.Lspan  = 80        # span length [km]
paramCh.alpha = 0.2        # fiber loss parameter [dB/km]
paramCh.D = 16             # fiber dispersion parameter [ps/nm/km]
paramCh.gamma = 1.3        # fiber nonlinear parameter [1/(W.km)]
paramCh.Fc = 193.1e12      # central optical frequency of the WDM spectrum
paramCh.hz = 0.5           # step-size of the split-step Fourier method [km]
paramCh.prgsBar = True     # show progress bar
paramCh.Fs = Rs*SpS        # sampling rate
paramCh.amp = 'edfa'       # type of amplifiers
paramCh.NF = 4.5           # Noise Figure of each amplirifer [dB]
#paramCh.seed = 456


# ------------------------------------------------
# Receiver Parameters
#-------------------------------------------------
```

```
# 1) local oscillator (LO) parameters:
FO  = -128e6                  # frequency offset
paramLO = parameters()
paramLO.P =2               # power in dBm
paramLO.lw = 100e3           # laser linewidth
paramLO.RIN_var = 0
paramLO.Fs = Fs
paramLO.seed = 789 # random seed for noise generation
paramLO.freqShift = 0 + FO  # downshift of the channel to be demodulated add␣
 ↪frequency offset

# 2) Front-End Parameters and photodiode paramters

# Frontend parameters
paramFE = parameters()
paramFE.Fs = Fs

# Photodiodes parameters
paramPD = parameters()
paramPD.B = Rs
paramPD.Fs = Fs
paramPD.ideal = True
paramPD.seed = 1011

# 3) Pulseshaping in the reciever using rrc filter
paramRxPulse = parameters()
paramRxPulse.SpS = SpS
paramRxPulse.nFilterTaps = nFilterTaps
paramRxPulse.rollOff = rollOff
paramRxPulse.pulseType = "rrc"

# 4) Decimation Parameters
paramDec = parameters()
paramDec.SpSin  = SpS
paramDec.SpSout = 2

# 5) Chromatic Dispersion Parameters
paramEDC = parameters()
paramEDC.L = paramCh.Ltotal
paramEDC.D = paramCh.D
paramEDC.Fc = paramCh.Fc
paramEDC.Rs = Rs
paramEDC.Fs = 2*Rs

# 6) Adaptive Equalization Parameters
paramEq = parameters()
```

```
paramEq.nTaps = 35
paramEq.SpS = paramDec.SpSout
paramEq.numIter = 2
paramEq.storeCoeff = False
paramEq.M = M
paramEq.shapingFactor = 0
paramEq.constType = "qam"
paramEq.prgsBar = False

# 7) Data-Aided or Blind Reciever Equalization
# Can be set here or not
#Data_Aided = True

# 8) Carrier and Phase recovery parameters using bps
paramCPR = parameters()
paramCPR.alg = 'bps'
paramCPR.M   = M
paramCPR.constType ="qam"
paramCPR.shapingFactor = 0
paramCPR.N   = 25
paramCPR.B   = 64
paramCPR.returnPhases = True
paramCPR.Ts = 1/Rs
```

[5]:
```
# ------------------------------------------------
# Simulation of the Optical System
# ------------------------------------------------

def simulate_optical_system(symbTx, paramSymb, paramPulse,paramIQM,sigLO,␣
 ↪paramCh, paramLO, paramFE, paramPD,paramRxPulse,paramDec
                             ,paramEDC,paramEq,paramCPR,␣
 ↪PA_enable=True,Data_Aided=True):

    """
   This function simulates the optical transmission of a single-polarization,
   single-channel 16/4-QAM signal.

    Parameters
    ----------
    Uses Global Parameters defined above

    Returns
    -------
    y_CPR_1 :
        Received signal at end of DSP chain (after CPR)
    d:
        Noemlaized Symbol Synchronization with the SymbTx. Used in calculations.
```

```
    """

    #␣
↪-----------------------------------------------------------------------------------------
    # TRANSMITTER

    # 2) Upsampling + pulse shaping
    pulse = pulseShape(paramPulse)
    symbolsUp = upsample(symbTx, SpS)
    sigTx = firFilter(pulse, symbolsUp)

    # 3) Choose nominal small-signal PA gain so the nominal drive is around␣
↪mzmScale * Vpi
    target_peak_V = mzmScale * Vpi
    peak_sigTx = np.max(np.abs(sigTx))


    if peak_sigTx == 0:
        raise ValueError("sigTx peak is zero; cannot set PA gain.")

    gain_linear = target_peak_V / peak_sigTx

    # 4) Driver amplifier / PA output directly in volts
    if PA_enable:
        u_drive, paInfo = pa_model_physical(
            sigTx,
            Fs=Fs,
            gain_linear=gain_linear,
            BLPF_enable = BLPF_enable,
            BO_dB=PA_BO_dB,
            p=PA_p,
            f3dB=PA_f3dB,
            order=3
        )
    else:
        u_drive = gain_linear * sigTx
        paInfo = {
            "gain_linear": gain_linear,
            "Vrms_in_V": np.sqrt(np.mean(np.abs(u_drive) ** 2)),
            "Vsat_V": None,
            "BO_dB": None,
            "p": None,
            "f3dB_Hz": None,
            "peak_out_V": np.max(np.abs(u_drive)),
            "rms_out_V": np.sqrt(np.mean(np.abs(u_drive) ** 2)),
        }
```

```
    # 5) IQ modulation: PA output drives the IQM directly
    sigTxo = iqm(sigLO, u_drive, paramIQM)

    # 6) Set launched optical power
    P_launch_W = dBm2W(P_launch_dBm)
    sigTxo = np.sqrt(P_launch_W) * pnorm(sigTxo)

    # End of Transmitter
    #␣
↪-------------------------------------------------------------------------------------------

    #␣
↪-------------------------------------------------------------------------------------------
    # CHANNEL

    sigCh = ssfm(sigTxo, paramCh)

    # End of CHANNEL
    #␣
↪-------------------------------------------------------------------------------------------

    #␣
↪-------------------------------------------------------------------------------------------
    # RECEIVER

    # 1) Generate CW laser LO field
    # Parameter which might change so needs to be in the function
    paramLO.Ns = len(sigCh)
    sigLO_Rx = basicLaserModel(paramLO)

    # 2) Coherent Reciever for single-polarization.
    # Contains a 90 hybrid, two balanced photodiodes and iqmixing
    sigRxFrontEnd = coherentReceiver(sigCh, sigLO_Rx, paramFE, paramPD)

    # 3) Pusleshaping Function
    pulse = pulseShape(paramRxPulse)
    sigRxPulseShape = firFilter(pulse, sigRxFrontEnd)

    # 4) Decimation
    sigRxDecimation = decimate(sigRxPulseShape, paramDec)

    # 5) Chromatic Dispersion Compensation for non-linear channel
    sigRxCD = edc(sigRxDecimation, paramEDC)

    # 6) Symbol Synchronization with the SymbTx
    symbRxCD = symbolSync(sigRxCD, symbTx, 2)
```

```
    # 7) Power Normalization
    x = pnorm(sigRxCD)
    d = pnorm(symbRxCD)

    # 8) Adaptive Equalization Parameter which might change
    paramEq.L = [int(0.2*d.shape[0]), int(0.8*d.shape[0])]

    # 9) Data Aided or Blind
    if Data_Aided:
        if M == 4:
            paramEq.alg = ['cma','cma'] # QPSK
            paramEq.mu = [5e-3, 1e-3]
            y_EQ = mimoAdaptEqualizer(x, paramEq, d)
        else:
            paramEq.alg = ['da-rde','rde'] # M-QAM
            paramEq.mu = [5e-3, 5e-4]
            y_EQ = mimoAdaptEqualizer(x, paramEq, d)
    else:
        if M == 4:
            paramEq.alg = ['cma','cma'] # QPSK
            paramEq.mu = [5e-3, 1e-3]
            y_EQ = mimoAdaptEqualizer(x, paramEq, None)
        else:
            paramEq.alg = ['cma','rde'] # M-QAM
            paramEq.mu = [5e-3, 1e-3]
            y_EQ = mimoAdaptEqualizer(x, paramEq, None)

    # 9) Carrier and Phase recovery using bps in the parameters
    y_CPR_1,   = cpr(y_EQ, paramCPR)

    return y_CPR_1,d
    # End of RECEIVER
    #␣
↪-----------------------------------------------------------------------------------------
```

[6]:
```
def perf_calc(symbTx, y_CPR_1, d, M, paramSymb, paramDec):

    """Performance Metric Exploration for Single Polarization
    """

    # remove phase ambiguity for 4-QAM/QPSK
    if M == 16:
        discard = 5000
        ind = np.arange(discard, len(symbTx) - discard)

        d = symbTx
```

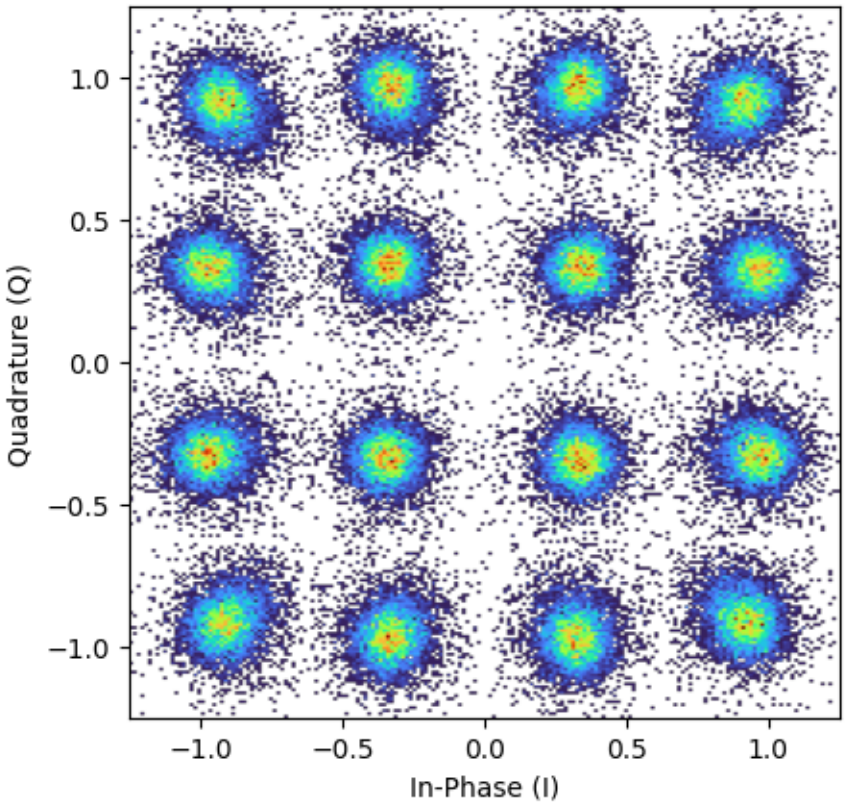
```
    # now compute metrics (both are 1 sample/symbol and same length)
        BER, SER, SNR = fastBERcalc(y_CPR_1[ind], d[ind], M, 'qam', px =␣
 ↪paramSymb.px)
        EVM = calcEVM(y_CPR_1[ind], M, 'qam', d[ind])
        Qfactor = ber2Qfactor(BER[0])

        print(' SER: %.3e,  '%(SER[0]))
        print(' BER: %.3e   '%(BER[0]))
        print(' SNR: %.3f dB'%(SNR[0]))
        print(' EVM: %.3f %%'%(EVM[0]*100))
        print(' Qfactor: %.3f,  '%(Qfactor))


    return BER
```

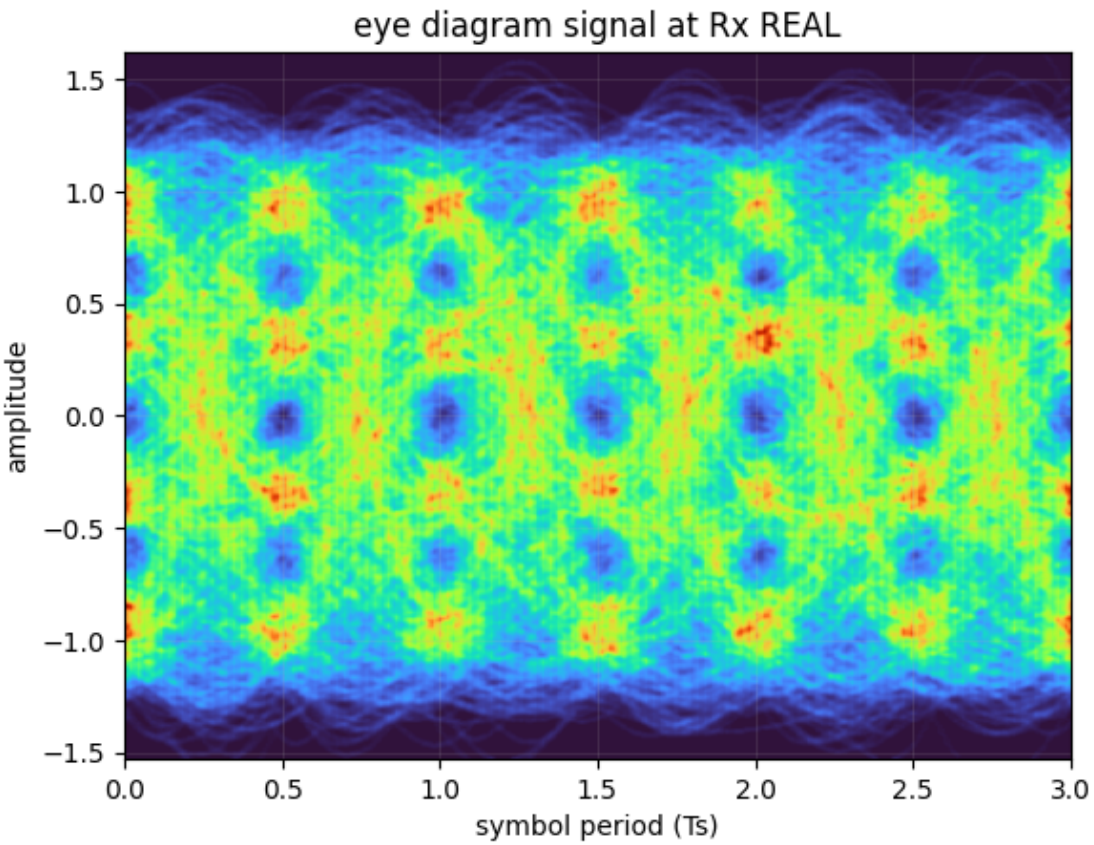
[7]:
```
# PA_BO_dB = 12 # Best BER at mzm 0.5 and Data Blind scenario - 3.6e-3
mzmScale = 0.66 # when you cross from here to 0.67 at a =3 thats when youll see␣
 ↪the BER weird issue on the data aided case jumping from 7.7e-2 to like 3e-3␣
 ↪suddenly without DPD
symbTx = symbolSource(paramSymb)

# Running the function to check that it works
y_CPR_1, d= simulate_optical_system(symbTx, paramSymb,␣
 ↪paramPulse,paramIQM,sigLO, paramCh, paramLO, paramFE,␣
 ↪paramPD,paramRxPulse,paramDec
                                ,paramEDC,paramEq,paramCPR, PA_enable=True,␣
 ↪Data_Aided=False)


perf_calc(symbTx, y_CPR_1, d, M, paramSymb, paramDec)


discard = 5000
# plot constellations
pconst(y_CPR_1[discard:-discard])

# plotting eye diagrams of sigTx
eyediagram(y_CPR_1.real[discard:-discard], y_CPR_1.real.size-2*discard,␣
 ↪paramDec.SpSout, plotlabel='signal at Rx REAL', ptype='fancy')
eyediagram(y_CPR_1.imag[discard:-discard], y_CPR_1.imag.size-2*discard,␣
 ↪paramDec.SpSout, plotlabel='signal at Rx IMAGNIARY', ptype='fancy')
```

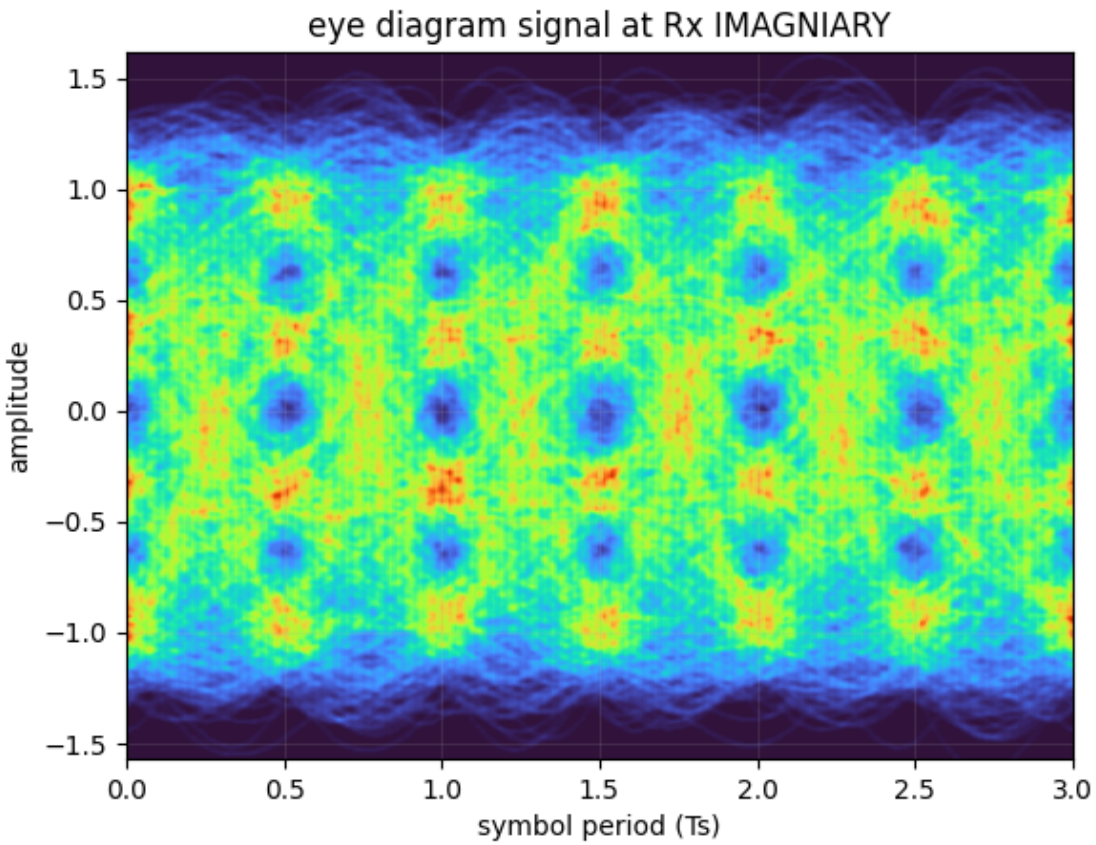
```
  0%|          | 0/1 [00:00<?, ?it/s]
Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
cma - training stage #0
```

```
cma pre-convergence training iteration #0
cma MSE = 0.431948.
cma pre-convergence training iteration #1
cma MSE = 0.425516.
rde - training stage #1
rde MSE = 0.024375.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 383.112 kHz
```

```
 SER: 1.642e-01,
 BER: 7.730e-02
 SNR: 5.529 dB
 EVM: 26.040 %
 Qfactor: 1.534,
```




eye diagram signal at Rx REAL
amplitude
symbol period (Ts)
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
0.0
0.5
1.0
1.5
2.0
2.5
3.0



0.1 DPD

[8]:
```
def build_model():
    inputs = layers.Input(shape=(None, 2)) # 2 for I and Q

    sec_a = layers.Conv1D(2, 100, padding='same')(inputs) # 100 taps was a␣
 ↪sweet spot, 20-ish fails to converge, tiker with different values.

#    sec_a = inputs

    nonlinear_1 = layers.Dense(20, activation=layers.LeakyReLU(0.1))(sec_a)
    nonlinear_2 = layers.Dense(20, activation=layers.LeakyReLU(0.
 ↪1))(nonlinear_1)
    nonlinear_3 = layers.Dense(2, activation=layers.LeakyReLU(0.1))(nonlinear_2)
    nonlinear_4 = layers.Dense(2, activation='linear')(nonlinear_3)


    # nonlinear_1 = layers.Dense(20, activation=tf.math.sin)(sec_a)
    # nonlinear_2 = layers.Dense(20, activation=tf.math.sin)(nonlinear_1)
    # nonlinear_3 = layers.Dense(2, activation=tf.math.sin)(nonlinear_2)
```

```
    # nonlinear_4 = layers.Dense(2, activation='linear')(nonlinear_3)




    outputs = layers.Add()([sec_a, nonlinear_4])

    # outputs = nonlinear_4

    return Model(inputs, outputs)

seq_length = 5000


# ILA
dpd_model = build_model()
dpd_model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-2),␣
 ↪loss='mse') # ILA
```

```
2026-03-25 14:23:03.930214: I metal_plugin/src/device/metal_device.cc:1154]
Metal device set to: Apple M3
2026-03-25 14:23:03.930241: I metal_plugin/src/device/metal_device.cc:296]
systemMemory: 24.00 GB
2026-03-25 14:23:03.930246: I metal_plugin/src/device/metal_device.cc:313]
maxCacheSize: 8.88 GB
2026-03-25 14:23:03.930260: I
tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:305]
Could not identify NUMA node of platform GPU ID 0, defaulting to 0. Your kernel
may not have been built with NUMA support.
2026-03-25 14:23:03.930268: I
tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:271]
Created TensorFlow device (/job:localhost/replica:0/task:0/device:GPU:0 with 0
MB memory) -> physical PluggableDevice (device: 0, name: METAL, pci bus id:
<undefined>)
```

[9]:
```
def split_i_q(arr):
    return np.stack([np.real(arr), np.imag(arr)]).T

def merge_i_q(arr):
    if arr.ndim == 3:
        return arr[:,:,0] + 1j*arr[:,:,1]
    elif arr.ndim == 2:
        return arr[:,0] + 1j*arr[:,1]
    else:
        raise ValueError("Input array must be 2D or 3D.")
```

[10]:
```
reshabe_len = len(symbTx)//seq_length

symbTx_nn = split_i_q(symbTx)
symbTx_nn = symbTx_nn[:reshabe_len*seq_length].reshape(-1, seq_length, 2) #␣
 ↪batching the symbols for training (shape: num_batches, seq_length,␣
 ↪num_features)

symbTx_nn.shape
```

[10]:
```
(20, 5000, 2)
```

[11]:
```
dpd_model.fit(symbTx_nn, symbTx_nn, epochs=500, verbose=1) # this line is␣
 ↪important, = starting as a passthrough.
```

```
Epoch 1/500

2026-03-25 14:23:04.324241: I
tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117]
Plugin optimizer for device_type GPU is enabled.

1/1             1s 806ms/step - loss:
1.0354
Epoch 2/500
1/1             0s 30ms/step - loss:
0.8634
Epoch 3/500
1/1             0s 24ms/step - loss:
0.7416
Epoch 4/500
1/1             0s 23ms/step - loss:
0.6518
Epoch 5/500
1/1             0s 23ms/step - loss:
0.5881
Epoch 6/500
1/1             0s 23ms/step - loss:
0.5455
Epoch 7/500
1/1             0s 25ms/step - loss:
0.5182
Epoch 8/500
1/1             0s 25ms/step - loss:
0.5008
Epoch 9/500
1/1             0s 27ms/step - loss:
0.4891
Epoch 10/500
1/1             0s 26ms/step - loss:
0.4803
```

```
Epoch 11/500
1/1             0s 23ms/step - loss:
0.4725
Epoch 12/500
1/1             0s 23ms/step - loss:
0.4646
Epoch 13/500
1/1             0s 23ms/step - loss:
0.4569
Epoch 14/500
1/1             0s 23ms/step - loss:
0.4493
Epoch 15/500
1/1             0s 23ms/step - loss:
0.4415
Epoch 16/500
1/1             0s 23ms/step - loss:
0.4331
Epoch 17/500
1/1             0s 24ms/step - loss:
0.4241
Epoch 18/500
1/1             0s 25ms/step - loss:
0.4146
Epoch 19/500
1/1             0s 26ms/step - loss:
0.4048
Epoch 20/500
1/1             0s 26ms/step - loss:
0.3953
Epoch 21/500
1/1             0s 27ms/step - loss:
0.3861
Epoch 22/500
1/1             0s 44ms/step - loss:
0.3773
Epoch 23/500
1/1             0s 29ms/step - loss:
0.3690
Epoch 24/500
1/1             0s 26ms/step - loss:
0.3611
Epoch 25/500
1/1             0s 23ms/step - loss:
0.3534
Epoch 26/500
1/1             0s 23ms/step - loss:
0.3459
```

```
Epoch 27/500
1/1             0s 23ms/step - loss:
0.3385
Epoch 28/500
1/1             0s 23ms/step - loss:
0.3312
Epoch 29/500
1/1             0s 24ms/step - loss:
0.3239
Epoch 30/500
1/1             0s 25ms/step - loss:
0.3167
Epoch 31/500
1/1             0s 28ms/step - loss:
0.3097
Epoch 32/500
1/1             0s 26ms/step - loss:
0.3030
Epoch 33/500
1/1             0s 22ms/step - loss:
0.2963
Epoch 34/500
1/1             0s 22ms/step - loss:
0.2897
Epoch 35/500
1/1             0s 23ms/step - loss:
0.2833
Epoch 36/500
1/1             0s 23ms/step - loss:
0.2769
Epoch 37/500
1/1             0s 24ms/step - loss:
0.2703
Epoch 38/500
1/1             0s 23ms/step - loss:
0.2636
Epoch 39/500
1/1             0s 24ms/step - loss:
0.2568
Epoch 40/500
1/1             0s 25ms/step - loss:
0.2499
Epoch 41/500
1/1             0s 26ms/step - loss:
0.2428
Epoch 42/500
1/1             0s 27ms/step - loss:
0.2354
```

```
Epoch 43/500
1/1             0s 22ms/step - loss:
0.2274
Epoch 44/500
1/1             0s 30ms/step - loss:
0.2188
Epoch 45/500
1/1             0s 23ms/step - loss:
0.2094
Epoch 46/500
1/1             0s 23ms/step - loss:
0.1993
Epoch 47/500
1/1             0s 23ms/step - loss:
0.1885
Epoch 48/500
1/1             0s 23ms/step - loss:
0.1772
Epoch 49/500
1/1             0s 46ms/step - loss:
0.1660
Epoch 50/500
1/1             0s 27ms/step - loss:
0.1552
Epoch 51/500
1/1             0s 29ms/step - loss:
0.1450
Epoch 52/500
1/1             0s 29ms/step - loss:
0.1357
Epoch 53/500
1/1             0s 28ms/step - loss:
0.1276
Epoch 54/500
1/1             0s 25ms/step - loss:
0.1211
Epoch 55/500
1/1             0s 26ms/step - loss:
0.1159
Epoch 56/500
1/1             0s 26ms/step - loss:
0.1117
Epoch 57/500
1/1             0s 24ms/step - loss:
0.1088
Epoch 58/500
1/1             0s 24ms/step - loss:
0.1066
```

```
Epoch 59/500
1/1             0s 23ms/step - loss:
0.1042
Epoch 60/500
1/1             0s 24ms/step - loss:
0.1008
Epoch 61/500
1/1             0s 24ms/step - loss:
0.0959
Epoch 62/500
1/1             0s 24ms/step - loss:
0.0896
Epoch 63/500
1/1             0s 26ms/step - loss:
0.0827
Epoch 64/500
1/1             0s 25ms/step - loss:
0.0760
Epoch 65/500
1/1             0s 26ms/step - loss:
0.0699
Epoch 66/500
1/1             0s 27ms/step - loss:
0.0647
Epoch 67/500
1/1             0s 21ms/step - loss:
0.0603
Epoch 68/500
1/1             0s 25ms/step - loss:
0.0563
Epoch 69/500
1/1             0s 24ms/step - loss:
0.0526
Epoch 70/500
1/1             0s 25ms/step - loss:
0.0491
Epoch 71/500
1/1             0s 24ms/step - loss:
0.0456
Epoch 72/500
1/1             0s 29ms/step - loss:
0.0423
Epoch 73/500
1/1             0s 45ms/step - loss:
0.0389
Epoch 74/500
1/1             0s 25ms/step - loss:
0.0357
```

```
Epoch 75/500
1/1             0s 26ms/step - loss:
0.0326
Epoch 76/500
1/1             0s 24ms/step - loss:
0.0298
Epoch 77/500
1/1             0s 25ms/step - loss:
0.0273
Epoch 78/500
1/1             0s 24ms/step - loss:
0.0250
Epoch 79/500
1/1             0s 24ms/step - loss:
0.0229
Epoch 80/500
1/1             0s 26ms/step - loss:
0.0209
Epoch 81/500
1/1             0s 27ms/step - loss:
0.0191
Epoch 82/500
1/1             0s 24ms/step - loss:
0.0175
Epoch 83/500
1/1             0s 26ms/step - loss:
0.0159
Epoch 84/500
1/1             0s 27ms/step - loss:
0.0145
Epoch 85/500
1/1             0s 25ms/step - loss:
0.0132
Epoch 86/500
1/1             0s 28ms/step - loss:
0.0119
Epoch 87/500
1/1             0s 27ms/step - loss:
0.0106
Epoch 88/500
1/1             0s 25ms/step - loss:
0.0094
Epoch 89/500
1/1             0s 26ms/step - loss:
0.0083
Epoch 90/500
1/1             0s 26ms/step - loss:
0.0074
```

```
Epoch 91/500
1/1             0s 24ms/step - loss:
0.0065
Epoch 92/500
1/1             0s 27ms/step - loss:
0.0056
Epoch 93/500
1/1             0s 24ms/step - loss:
0.0048
Epoch 94/500
1/1             0s 25ms/step - loss:
0.0041
Epoch 95/500
1/1             0s 26ms/step - loss:
0.0035
Epoch 96/500
1/1             0s 24ms/step - loss:
0.0030
Epoch 97/500
1/1             0s 24ms/step - loss:
0.0026
Epoch 98/500
1/1             0s 24ms/step - loss:
0.0021
Epoch 99/500
1/1             0s 24ms/step - loss:
0.0017
Epoch 100/500
1/1             0s 42ms/step - loss:
0.0013
Epoch 101/500
1/1             0s 29ms/step - loss:
9.8332e-04
Epoch 102/500
1/1             0s 24ms/step - loss:
7.9009e-04
Epoch 103/500
1/1             0s 25ms/step - loss:
6.7969e-04
Epoch 104/500
1/1             0s 24ms/step - loss:
6.0825e-04
Epoch 105/500
1/1             0s 24ms/step - loss:
5.4873e-04
Epoch 106/500
1/1             0s 25ms/step - loss:
4.8659e-04
```

```
Epoch 107/500
1/1             0s 26ms/step - loss:
4.3512e-04
Epoch 108/500
1/1             0s 25ms/step - loss:
4.1220e-04
Epoch 109/500
1/1             0s 25ms/step - loss:
4.2595e-04
Epoch 110/500
1/1             0s 24ms/step - loss:
4.6777e-04
Epoch 111/500
1/1             0s 24ms/step - loss:
5.1364e-04
Epoch 112/500
1/1             0s 24ms/step - loss:
5.4479e-04
Epoch 113/500
1/1             0s 24ms/step - loss:
5.5203e-04
Epoch 114/500
1/1             0s 25ms/step - loss:
5.4413e-04
Epoch 115/500
1/1             0s 25ms/step - loss:
5.3260e-04
Epoch 116/500
1/1             0s 26ms/step - loss:
5.2220e-04
Epoch 117/500
1/1             0s 28ms/step - loss:
5.1269e-04
Epoch 118/500
1/1             0s 22ms/step - loss:
5.0223e-04
Epoch 119/500
1/1             0s 26ms/step - loss:
4.8418e-04
Epoch 120/500
1/1             0s 24ms/step - loss:
4.5665e-04
Epoch 121/500
1/1             0s 26ms/step - loss:
4.2136e-04
Epoch 122/500
1/1             0s 25ms/step - loss:
3.8609e-04
```

```
Epoch 123/500
1/1             0s 47ms/step - loss:
3.5513e-04
Epoch 124/500
1/1             0s 26ms/step - loss:
3.3159e-04
Epoch 125/500
1/1             0s 23ms/step - loss:
3.1035e-04
Epoch 126/500
1/1             0s 23ms/step - loss:
2.8775e-04
Epoch 127/500
1/1             0s 23ms/step - loss:
2.6427e-04
Epoch 128/500
1/1             0s 23ms/step - loss:
2.4180e-04
Epoch 129/500
1/1             0s 24ms/step - loss:
2.2130e-04
Epoch 130/500
1/1             0s 24ms/step - loss:
2.0347e-04
Epoch 131/500
1/1             0s 24ms/step - loss:
1.8816e-04
Epoch 132/500
1/1             0s 24ms/step - loss:
1.7507e-04
Epoch 133/500
1/1             0s 25ms/step - loss:
1.6365e-04
Epoch 134/500
1/1             0s 24ms/step - loss:
1.5388e-04
Epoch 135/500
1/1             0s 25ms/step - loss:
1.4697e-04
Epoch 136/500
1/1             0s 24ms/step - loss:
1.4269e-04
Epoch 137/500
1/1             0s 25ms/step - loss:
1.4002e-04
Epoch 138/500
1/1             0s 24ms/step - loss:
1.3818e-04
```

```
Epoch 139/500
1/1             0s 24ms/step - loss:
1.3653e-04
Epoch 140/500
1/1             0s 23ms/step - loss:
1.3515e-04
Epoch 141/500
1/1             0s 24ms/step - loss:
1.3438e-04
Epoch 142/500
1/1             0s 24ms/step - loss:
1.3439e-04
Epoch 143/500
1/1             0s 49ms/step - loss:
1.3523e-04
Epoch 144/500
1/1             0s 28ms/step - loss:
1.3619e-04
Epoch 145/500
1/1             0s 25ms/step - loss:
1.3639e-04
Epoch 146/500
1/1             0s 22ms/step - loss:
1.3567e-04
Epoch 147/500
1/1             0s 27ms/step - loss:
1.3437e-04
Epoch 148/500
1/1             0s 28ms/step - loss:
1.3299e-04
Epoch 149/500
1/1             0s 22ms/step - loss:
1.3184e-04
Epoch 150/500
1/1             0s 27ms/step - loss:
1.3073e-04
Epoch 151/500
1/1             0s 25ms/step - loss:
1.2930e-04
Epoch 152/500
1/1             0s 25ms/step - loss:
1.2735e-04
Epoch 153/500
1/1             0s 25ms/step - loss:
1.2498e-04
Epoch 154/500
1/1             0s 24ms/step - loss:
1.2251e-04
```

```
Epoch 155/500
1/1             0s 24ms/step - loss:
1.2029e-04
Epoch 156/500
1/1             0s 23ms/step - loss:
1.1839e-04
Epoch 157/500
1/1             0s 24ms/step - loss:
1.1662e-04
Epoch 158/500
1/1             0s 25ms/step - loss:
1.1487e-04
Epoch 159/500
1/1             0s 25ms/step - loss:
1.1312e-04
Epoch 160/500
1/1             0s 27ms/step - loss:
1.1142e-04
Epoch 161/500
1/1             0s 22ms/step - loss:
1.0984e-04
Epoch 162/500
1/1             0s 28ms/step - loss:
1.0840e-04
Epoch 163/500
1/1             0s 29ms/step - loss:
1.0703e-04
Epoch 164/500
1/1             0s 25ms/step - loss:
1.0580e-04
Epoch 165/500
1/1             0s 30ms/step - loss:
1.0477e-04
Epoch 166/500
1/1             0s 26ms/step - loss:
1.0391e-04
Epoch 167/500
1/1             0s 29ms/step - loss:
1.0310e-04
Epoch 168/500
1/1             0s 29ms/step - loss:
1.0222e-04
Epoch 169/500
1/1             0s 52ms/step - loss:
1.0121e-04
Epoch 170/500
1/1             0s 23ms/step - loss:
1.0017e-04
```

```
Epoch 171/500
1/1             0s 22ms/step - loss:
9.9238e-05
Epoch 172/500
1/1             0s 23ms/step - loss:
9.8516e-05
Epoch 173/500
1/1             0s 23ms/step - loss:
9.7975e-05
Epoch 174/500
1/1             0s 23ms/step - loss:
9.7516e-05
Epoch 175/500
1/1             0s 24ms/step - loss:
9.7011e-05
Epoch 176/500
1/1             0s 23ms/step - loss:
9.6407e-05
Epoch 177/500
1/1             0s 25ms/step - loss:
9.5742e-05
Epoch 178/500
1/1             0s 24ms/step - loss:
9.5114e-05
Epoch 179/500
1/1             0s 25ms/step - loss:
9.4620e-05
Epoch 180/500
1/1             0s 25ms/step - loss:
9.4314e-05
Epoch 181/500
1/1             0s 25ms/step - loss:
9.4200e-05
Epoch 182/500
1/1             0s 25ms/step - loss:
9.4224e-05
Epoch 183/500
1/1             0s 25ms/step - loss:
9.4215e-05
Epoch 184/500
1/1             0s 24ms/step - loss:
9.3872e-05
Epoch 185/500
1/1             0s 24ms/step - loss:
9.3060e-05
Epoch 186/500
1/1             0s 24ms/step - loss:
9.2107e-05
```

```
Epoch 187/500
1/1             0s 24ms/step - loss:
9.1336e-05
Epoch 188/500
1/1             0s 25ms/step - loss:
9.0507e-05
Epoch 189/500
1/1             0s 25ms/step - loss:
8.9286e-05
Epoch 190/500
1/1             0s 25ms/step - loss:
8.8006e-05
Epoch 191/500
1/1             0s 26ms/step - loss:
8.7349e-05
Epoch 192/500
1/1             0s 43ms/step - loss:
8.7380e-05
Epoch 193/500
1/1             0s 27ms/step - loss:
8.7421e-05
Epoch 194/500
1/1             0s 25ms/step - loss:
8.6870e-05
Epoch 195/500
1/1             0s 26ms/step - loss:
8.5939e-05
Epoch 196/500
1/1             0s 27ms/step - loss:
8.5329e-05
Epoch 197/500
1/1             0s 25ms/step - loss:
8.5278e-05
Epoch 198/500
1/1             0s 26ms/step - loss:
8.5356e-05
Epoch 199/500
1/1             0s 26ms/step - loss:
8.5079e-05
Epoch 200/500
1/1             0s 24ms/step - loss:
8.4347e-05
Epoch 201/500
1/1             0s 25ms/step - loss:
8.3348e-05
Epoch 202/500
1/1             0s 24ms/step - loss:
8.2422e-05
```

```
Epoch 203/500
1/1             0s 25ms/step - loss:
8.1858e-05
Epoch 204/500
1/1             0s 24ms/step - loss:
8.1568e-05
Epoch 205/500
1/1             0s 24ms/step - loss:
8.1254e-05
Epoch 206/500
1/1             0s 24ms/step - loss:
8.0767e-05
Epoch 207/500
1/1             0s 24ms/step - loss:
8.0195e-05
Epoch 208/500
1/1             0s 23ms/step - loss:
7.9732e-05
Epoch 209/500
1/1             0s 24ms/step - loss:
7.9393e-05
Epoch 210/500
1/1             0s 24ms/step - loss:
7.9037e-05
Epoch 211/500
1/1             0s 24ms/step - loss:
7.8612e-05
Epoch 212/500
1/1             0s 26ms/step - loss:
7.8205e-05
Epoch 213/500
1/1             0s 42ms/step - loss:
7.7890e-05
Epoch 214/500
1/1             0s 30ms/step - loss:
7.7604e-05
Epoch 215/500
1/1             0s 26ms/step - loss:
7.7264e-05
Epoch 216/500
1/1             0s 22ms/step - loss:
7.6929e-05
Epoch 217/500
1/1             0s 23ms/step - loss:
7.6753e-05
Epoch 218/500
1/1             0s 23ms/step - loss:
7.6793e-05
```

```
Epoch 219/500
1/1             0s 23ms/step - loss:
7.6893e-05
Epoch 220/500
1/1             0s 23ms/step - loss:
7.6741e-05
Epoch 221/500
1/1             0s 23ms/step - loss:
7.6159e-05
Epoch 222/500
1/1             0s 24ms/step - loss:
7.5427e-05
Epoch 223/500
1/1             0s 25ms/step - loss:
7.5116e-05
Epoch 224/500
1/1             0s 27ms/step - loss:
7.5378e-05
Epoch 225/500
1/1             0s 25ms/step - loss:
7.5764e-05
Epoch 226/500
1/1             0s 26ms/step - loss:
7.5751e-05
Epoch 227/500
1/1             0s 25ms/step - loss:
7.5242e-05
Epoch 228/500
1/1             0s 26ms/step - loss:
7.4531e-05
Epoch 229/500
1/1             0s 27ms/step - loss:
7.3796e-05
Epoch 230/500
1/1             0s 25ms/step - loss:
7.2935e-05
Epoch 231/500
1/1             0s 26ms/step - loss:
7.1967e-05
Epoch 232/500
1/1             0s 26ms/step - loss:
7.1161e-05
Epoch 233/500
1/1             0s 24ms/step - loss:
7.0717e-05
Epoch 234/500
1/1             0s 43ms/step - loss:
7.0537e-05
```

```
Epoch 235/500
1/1             0s 27ms/step - loss:
7.0337e-05
Epoch 236/500
1/1             0s 28ms/step - loss:
6.9962e-05
Epoch 237/500
1/1             0s 26ms/step - loss:
6.9521e-05
Epoch 238/500
1/1             0s 24ms/step - loss:
6.9181e-05
Epoch 239/500
1/1             0s 24ms/step - loss:
6.8939e-05
Epoch 240/500
1/1             0s 24ms/step - loss:
6.8671e-05
Epoch 241/500
1/1             0s 24ms/step - loss:
6.8304e-05
Epoch 242/500
1/1             0s 24ms/step - loss:
6.7901e-05
Epoch 243/500
1/1             0s 23ms/step - loss:
6.7555e-05
Epoch 244/500
1/1             0s 24ms/step - loss:
6.7270e-05
Epoch 245/500
1/1             0s 24ms/step - loss:
6.6977e-05
Epoch 246/500
1/1             0s 25ms/step - loss:
6.6635e-05
Epoch 247/500
1/1             0s 25ms/step - loss:
6.6298e-05
Epoch 248/500
1/1             0s 27ms/step - loss:
6.6072e-05
Epoch 249/500
1/1             0s 28ms/step - loss:
6.6011e-05
Epoch 250/500
1/1             0s 26ms/step - loss:
6.6093e-05
```

```
Epoch 251/500
1/1             0s 25ms/step - loss:
6.6294e-05
Epoch 252/500
1/1             0s 24ms/step - loss:
6.6674e-05
Epoch 253/500
1/1             0s 43ms/step - loss:
6.7401e-05
Epoch 254/500
1/1             0s 25ms/step - loss:
6.8670e-05
Epoch 255/500
1/1             0s 24ms/step - loss:
7.0660e-05
Epoch 256/500
1/1             0s 25ms/step - loss:
7.3735e-05
Epoch 257/500
1/1             0s 25ms/step - loss:
7.8906e-05
Epoch 258/500
1/1             0s 25ms/step - loss:
8.7961e-05
Epoch 259/500
1/1             0s 26ms/step - loss:
1.0311e-04
Epoch 260/500
1/1             0s 27ms/step - loss:
1.2500e-04
Epoch 261/500
1/1             0s 25ms/step - loss:
1.5140e-04
Epoch 262/500
1/1             0s 26ms/step - loss:
1.7180e-04
Epoch 263/500
1/1             0s 27ms/step - loss:
1.7091e-04
Epoch 264/500
1/1             0s 22ms/step - loss:
1.4080e-04
Epoch 265/500
1/1             0s 25ms/step - loss:
9.6880e-05
Epoch 266/500
1/1             0s 24ms/step - loss:
6.7530e-05
```

```
Epoch 267/500
1/1             0s 24ms/step - loss:
6.7418e-05
Epoch 268/500
1/1             0s 25ms/step - loss:
8.7546e-05
Epoch 269/500
1/1             0s 24ms/step - loss:
1.0638e-04
Epoch 270/500
1/1             0s 27ms/step - loss:
1.0637e-04
Epoch 271/500
1/1             0s 45ms/step - loss:
8.7558e-05
Epoch 272/500
1/1             0s 27ms/step - loss:
6.6548e-05
Epoch 273/500
1/1             0s 29ms/step - loss:
5.9735e-05
Epoch 274/500
1/1             0s 29ms/step - loss:
6.8340e-05
Epoch 275/500
1/1             0s 27ms/step - loss:
8.0072e-05
Epoch 276/500
1/1             0s 25ms/step - loss:
8.2832e-05
Epoch 277/500
1/1             0s 26ms/step - loss:
7.4284e-05
Epoch 278/500
1/1             0s 25ms/step - loss:
6.2376e-05
Epoch 279/500
1/1             0s 26ms/step - loss:
5.5988e-05
Epoch 280/500
1/1             0s 26ms/step - loss:
5.8251e-05
Epoch 281/500
1/1             0s 24ms/step - loss:
6.5043e-05
Epoch 282/500
1/1             0s 24ms/step - loss:
6.9581e-05
```

```
Epoch 283/500
1/1             0s 24ms/step - loss:
6.7777e-05
Epoch 284/500
1/1             0s 24ms/step - loss:
6.1290e-05
Epoch 285/500
1/1             0s 24ms/step - loss:
5.5481e-05
Epoch 286/500
1/1             0s 41ms/step - loss:
5.3866e-05
Epoch 287/500
1/1             0s 27ms/step - loss:
5.5854e-05
Epoch 288/500
1/1             0s 25ms/step - loss:
5.8705e-05
Epoch 289/500
1/1             0s 27ms/step - loss:
6.0193e-05
Epoch 290/500
1/1             0s 28ms/step - loss:
5.9743e-05
Epoch 291/500
1/1             0s 25ms/step - loss:
5.7397e-05
Epoch 292/500
1/1             0s 27ms/step - loss:
5.4350e-05
Epoch 293/500
1/1             0s 28ms/step - loss:
5.2259e-05
Epoch 294/500
1/1             0s 26ms/step - loss:
5.2169e-05
Epoch 295/500
1/1             0s 25ms/step - loss:
5.3508e-05
Epoch 296/500
1/1             0s 25ms/step - loss:
5.4850e-05
Epoch 297/500
1/1             0s 24ms/step - loss:
5.5260e-05
Epoch 298/500
1/1             0s 24ms/step - loss:
5.4752e-05
```

```
Epoch 299/500
1/1             0s 23ms/step - loss:
5.3698e-05
Epoch 300/500
1/1             0s 24ms/step - loss:
5.2441e-05
Epoch 301/500
1/1             0s 24ms/step - loss:
5.1210e-05
Epoch 302/500
1/1             0s 24ms/step - loss:
5.0267e-05
Epoch 303/500
1/1             0s 49ms/step - loss:
4.9833e-05
Epoch 304/500
1/1             0s 28ms/step - loss:
4.9905e-05
Epoch 305/500
1/1             0s 26ms/step - loss:
5.0264e-05
Epoch 306/500
1/1             0s 25ms/step - loss:
5.0673e-05
Epoch 307/500
1/1             0s 25ms/step - loss:
5.0899e-05
Epoch 308/500
1/1             0s 24ms/step - loss:
5.0838e-05
Epoch 309/500
1/1             0s 24ms/step - loss:
5.0439e-05
Epoch 310/500
1/1             0s 23ms/step - loss:
4.9821e-05
Epoch 311/500
1/1             0s 24ms/step - loss:
4.9129e-05
Epoch 312/500
1/1             0s 24ms/step - loss:
4.8489e-05
Epoch 313/500
1/1             0s 24ms/step - loss:
4.7965e-05
Epoch 314/500
1/1             0s 25ms/step - loss:
4.7566e-05
```

```
Epoch 315/500
1/1             0s 25ms/step - loss:
4.7303e-05
Epoch 316/500
1/1             0s 27ms/step - loss:
4.7143e-05
Epoch 317/500
1/1             0s 28ms/step - loss:
4.7037e-05
Epoch 318/500
1/1             0s 26ms/step - loss:
4.6955e-05
Epoch 319/500
1/1             0s 24ms/step - loss:
4.6875e-05
Epoch 320/500
1/1             0s 24ms/step - loss:
4.6825e-05
Epoch 321/500
1/1             0s 24ms/step - loss:
4.6836e-05
Epoch 322/500
1/1             0s 23ms/step - loss:
4.6952e-05
Epoch 323/500
1/1             0s 23ms/step - loss:
4.7185e-05
Epoch 324/500
1/1             0s 24ms/step - loss:
4.7584e-05
Epoch 325/500
1/1             0s 44ms/step - loss:
4.8227e-05
Epoch 326/500
1/1             0s 27ms/step - loss:
4.9254e-05
Epoch 327/500
1/1             0s 29ms/step - loss:
5.0872e-05
Epoch 328/500
1/1             0s 24ms/step - loss:
5.3472e-05
Epoch 329/500
1/1             0s 23ms/step - loss:
5.7493e-05
Epoch 330/500
1/1             0s 23ms/step - loss:
6.3885e-05
```

```
Epoch 331/500
1/1             0s 22ms/step - loss:
7.3655e-05
Epoch 332/500
1/1             0s 24ms/step - loss:
8.8573e-05
Epoch 333/500
1/1             0s 24ms/step - loss:
1.0978e-04
Epoch 334/500
1/1             0s 24ms/step - loss:
1.3762e-04
Epoch 335/500
1/1             0s 25ms/step - loss:
1.6737e-04
Epoch 336/500
1/1             0s 27ms/step - loss:
1.8834e-04
Epoch 337/500
1/1             0s 29ms/step - loss:
1.8393e-04
Epoch 338/500
1/1             0s 26ms/step - loss:
1.4739e-04
Epoch 339/500
1/1             0s 47ms/step - loss:
9.3131e-05
Epoch 340/500
1/1             0s 27ms/step - loss:
5.3234e-05
Epoch 341/500
1/1             0s 29ms/step - loss:
4.8163e-05
Epoch 342/500
1/1             0s 22ms/step - loss:
7.1306e-05
Epoch 343/500
1/1             0s 28ms/step - loss:
9.8259e-05
Epoch 344/500
1/1             0s 26ms/step - loss:
1.0608e-04
Epoch 345/500
1/1             0s 23ms/step - loss:
8.9141e-05
Epoch 346/500
1/1             0s 23ms/step - loss:
6.1593e-05
```

```
Epoch 347/500
1/1             0s 23ms/step - loss:
4.4743e-05
Epoch 348/500
1/1             0s 23ms/step - loss:
4.7694e-05
Epoch 349/500
1/1             0s 24ms/step - loss:
6.2427e-05
Epoch 350/500
1/1             0s 25ms/step - loss:
7.3587e-05
Epoch 351/500
1/1             0s 26ms/step - loss:
7.1369e-05
Epoch 352/500
1/1             0s 25ms/step - loss:
5.8483e-05
Epoch 353/500
1/1             0s 42ms/step - loss:
4.5731e-05
Epoch 354/500
1/1             0s 26ms/step - loss:
4.2108e-05
Epoch 355/500
1/1             0s 28ms/step - loss:
4.7675e-05
Epoch 356/500
1/1             0s 29ms/step - loss:
5.5377e-05
Epoch 357/500
1/1             0s 24ms/step - loss:
5.8037e-05
Epoch 358/500
1/1             0s 25ms/step - loss:
5.3792e-05
Epoch 359/500
1/1             0s 26ms/step - loss:
4.6490e-05
Epoch 360/500
1/1             0s 25ms/step - loss:
4.1490e-05
Epoch 361/500
1/1             0s 25ms/step - loss:
4.1357e-05
Epoch 362/500
1/1             0s 26ms/step - loss:
4.4681e-05
```

```
Epoch 363/500
1/1             0s 25ms/step - loss:
4.8058e-05
Epoch 364/500
1/1             0s 25ms/step - loss:
4.8873e-05
Epoch 365/500
1/1             0s 24ms/step - loss:
4.6654e-05
Epoch 366/500
1/1             0s 24ms/step - loss:
4.3104e-05
Epoch 367/500
1/1             0s 45ms/step - loss:
4.0296e-05
Epoch 368/500
1/1             0s 26ms/step - loss:
3.9586e-05
Epoch 369/500
1/1             0s 25ms/step - loss:
4.0793e-05
Epoch 370/500
1/1             0s 26ms/step - loss:
4.2618e-05
Epoch 371/500
1/1             0s 27ms/step - loss:
4.3747e-05
Epoch 372/500
1/1             0s 25ms/step - loss:
4.3434e-05
Epoch 373/500
1/1             0s 26ms/step - loss:
4.2036e-05
Epoch 374/500
1/1             0s 26ms/step - loss:
4.0274e-05
Epoch 375/500
1/1             0s 25ms/step - loss:
3.9009e-05
Epoch 376/500
1/1             0s 25ms/step - loss:
3.8611e-05
Epoch 377/500
1/1             0s 24ms/step - loss:
3.8946e-05
Epoch 378/500
1/1             0s 24ms/step - loss:
3.9555e-05
```

```
Epoch 379/500
1/1             0s 24ms/step - loss:
3.9959e-05
Epoch 380/500
1/1             0s 24ms/step - loss:
3.9922e-05
Epoch 381/500
1/1             0s 25ms/step - loss:
3.9439e-05
Epoch 382/500
1/1             0s 25ms/step - loss:
3.8687e-05
Epoch 383/500
1/1             0s 27ms/step - loss:
3.7881e-05
Epoch 384/500
1/1             0s 22ms/step - loss:
3.7204e-05
Epoch 385/500
1/1             0s 28ms/step - loss:
3.6750e-05
Epoch 386/500
1/1             0s 26ms/step - loss:
3.6528e-05
Epoch 387/500
1/1             0s 45ms/step - loss:
3.6487e-05
Epoch 388/500
1/1             0s 23ms/step - loss:
3.6548e-05
Epoch 389/500
1/1             0s 23ms/step - loss:
3.6618e-05
Epoch 390/500
1/1             0s 23ms/step - loss:
3.6642e-05
Epoch 391/500
1/1             0s 24ms/step - loss:
3.6576e-05
Epoch 392/500
1/1             0s 25ms/step - loss:
3.6413e-05
Epoch 393/500
1/1             0s 28ms/step - loss:
3.6172e-05
Epoch 394/500
1/1             0s 26ms/step - loss:
3.5870e-05
```

```
Epoch 395/500
1/1             0s 23ms/step - loss:
3.5532e-05
Epoch 396/500
1/1             0s 22ms/step - loss:
3.5191e-05
Epoch 397/500
1/1             0s 23ms/step - loss:
3.4861e-05
Epoch 398/500
1/1             0s 24ms/step - loss:
3.4571e-05
Epoch 399/500
1/1             0s 24ms/step - loss:
3.4333e-05
Epoch 400/500
1/1             0s 45ms/step - loss:
3.4177e-05
Epoch 401/500
1/1             0s 27ms/step - loss:
3.4123e-05
Epoch 402/500
1/1             0s 26ms/step - loss:
3.4198e-05
Epoch 403/500
1/1             0s 27ms/step - loss:
3.4397e-05
Epoch 404/500
1/1             0s 28ms/step - loss:
3.4648e-05
Epoch 405/500
1/1             0s 22ms/step - loss:
3.4815e-05
Epoch 406/500
1/1             0s 28ms/step - loss:
3.4744e-05
Epoch 407/500
1/1             0s 25ms/step - loss:
3.4517e-05
Epoch 408/500
1/1             0s 23ms/step - loss:
3.4470e-05
Epoch 409/500
1/1             0s 22ms/step - loss:
3.5021e-05
Epoch 410/500
1/1             0s 23ms/step - loss:
3.6400e-05
```

```
Epoch 411/500
1/1             0s 27ms/step - loss:
3.8827e-05
Epoch 412/500
1/1             0s 23ms/step - loss:
4.2731e-05
Epoch 413/500
1/1             0s 24ms/step - loss:
4.9087e-05
Epoch 414/500
1/1             0s 26ms/step - loss:
5.9326e-05
Epoch 415/500
1/1             0s 28ms/step - loss:
7.5673e-05
Epoch 416/500
1/1             0s 21ms/step - loss:
1.0023e-04
Epoch 417/500
1/1             0s 28ms/step - loss:
1.3588e-04
Epoch 418/500
1/1             0s 44ms/step - loss:
1.7996e-04
Epoch 419/500
1/1             0s 28ms/step - loss:
2.2620e-04
Epoch 420/500
1/1             0s 29ms/step - loss:
2.4862e-04
Epoch 421/500
1/1             0s 24ms/step - loss:
2.2840e-04
Epoch 422/500
1/1             0s 23ms/step - loss:
1.5585e-04
Epoch 423/500
1/1             0s 22ms/step - loss:
7.3885e-05
Epoch 424/500
1/1             0s 23ms/step - loss:
3.1319e-05
Epoch 425/500
1/1             0s 23ms/step - loss:
4.6145e-05
Epoch 426/500
1/1             0s 24ms/step - loss:
9.1147e-05
```

```
Epoch 427/500
1/1             0s 25ms/step - loss:
1.2134e-04
Epoch 428/500
1/1             0s 26ms/step - loss:
1.1178e-04
Epoch 429/500
1/1             0s 24ms/step - loss:
7.0820e-05
Epoch 430/500
1/1             0s 25ms/step - loss:
3.5742e-05
Epoch 431/500
1/1             0s 24ms/step - loss:
3.1534e-05
Epoch 432/500
1/1             0s 24ms/step - loss:
5.2584e-05
Epoch 433/500
1/1             0s 24ms/step - loss:
7.3075e-05
Epoch 434/500
1/1             0s 47ms/step - loss:
7.2119e-05
Epoch 435/500
1/1             0s 27ms/step - loss:
5.2351e-05
Epoch 436/500
1/1             0s 28ms/step - loss:
3.2291e-05
Epoch 437/500
1/1             0s 29ms/step - loss:
2.7938e-05
Epoch 438/500
1/1             0s 26ms/step - loss:
3.8212e-05
Epoch 439/500
1/1             0s 26ms/step - loss:
4.9609e-05
Epoch 440/500
1/1             0s 26ms/step - loss:
5.0496e-05
Epoch 441/500
1/1             0s 23ms/step - loss:
4.0587e-05
Epoch 442/500
1/1             0s 26ms/step - loss:
2.9419e-05
```

```
Epoch 443/500
1/1             0s 27ms/step - loss:
2.5729e-05
Epoch 444/500
1/1             0s 25ms/step - loss:
3.0166e-05
Epoch 445/500
1/1             0s 25ms/step - loss:
3.6474e-05
Epoch 446/500
1/1             0s 25ms/step - loss:
3.8205e-05
Epoch 447/500
1/1             0s 25ms/step - loss:
3.4144e-05
Epoch 448/500
1/1             0s 26ms/step - loss:
2.7974e-05
Epoch 449/500
1/1             0s 24ms/step - loss:
2.4461e-05
Epoch 450/500
1/1             0s 25ms/step - loss:
2.5268e-05
Epoch 451/500
1/1             0s 24ms/step - loss:
2.8389e-05
Epoch 452/500
1/1             0s 24ms/step - loss:
3.0518e-05
Epoch 453/500
1/1             0s 24ms/step - loss:
2.9784e-05
Epoch 454/500
1/1             0s 24ms/step - loss:
2.6976e-05
Epoch 455/500
1/1             0s 43ms/step - loss:
2.4202e-05
Epoch 456/500
1/1             0s 26ms/step - loss:
2.3117e-05
Epoch 457/500
1/1             0s 28ms/step - loss:
2.3793e-05
Epoch 458/500
1/1             0s 29ms/step - loss:
2.5102e-05
```

```
Epoch 459/500
1/1             0s 22ms/step - loss:
2.5806e-05
Epoch 460/500
1/1             0s 28ms/step - loss:
2.5391e-05
Epoch 461/500
1/1             0s 28ms/step - loss:
2.4181e-05
Epoch 462/500
1/1             0s 25ms/step - loss:
2.2890e-05
Epoch 463/500
1/1             0s 26ms/step - loss:
2.2090e-05
Epoch 464/500
1/1             0s 27ms/step - loss:
2.1953e-05
Epoch 465/500
1/1             0s 25ms/step - loss:
2.2274e-05
Epoch 466/500
1/1             0s 26ms/step - loss:
2.2673e-05
Epoch 467/500
1/1             0s 27ms/step - loss:
2.2825e-05
Epoch 468/500
1/1             0s 25ms/step - loss:
2.2588e-05
Epoch 469/500
1/1             0s 25ms/step - loss:
2.2048e-05
Epoch 470/500
1/1             0s 24ms/step - loss:
2.1410e-05
Epoch 471/500
1/1             0s 24ms/step - loss:
2.0899e-05
Epoch 472/500
1/1             0s 24ms/step - loss:
2.0631e-05
Epoch 473/500
1/1             0s 44ms/step - loss:
2.0600e-05
Epoch 474/500
1/1             0s 26ms/step - loss:
2.0703e-05
```

```
Epoch 475/500
1/1             0s 28ms/step - loss:
2.0806e-05
Epoch 476/500
1/1             0s 27ms/step - loss:
2.0814e-05
Epoch 477/500
1/1             0s 25ms/step - loss:
2.0686e-05
Epoch 478/500
1/1             0s 26ms/step - loss:
2.0451e-05
Epoch 479/500
1/1             0s 26ms/step - loss:
2.0160e-05
Epoch 480/500
1/1             0s 25ms/step - loss:
1.9880e-05
Epoch 481/500
1/1             0s 25ms/step - loss:
1.9645e-05
Epoch 482/500
1/1             0s 24ms/step - loss:
1.9476e-05
Epoch 483/500
1/1             0s 25ms/step - loss:
1.9367e-05
Epoch 484/500
1/1             0s 25ms/step - loss:
1.9308e-05
Epoch 485/500
1/1             0s 27ms/step - loss:
1.9286e-05
Epoch 486/500
1/1             0s 28ms/step - loss:
1.9285e-05
Epoch 487/500
1/1             0s 44ms/step - loss:
1.9314e-05
Epoch 488/500
1/1             0s 30ms/step - loss:
1.9385e-05
Epoch 489/500
1/1             0s 26ms/step - loss:
1.9515e-05
Epoch 490/500
1/1             0s 27ms/step - loss:
1.9704e-05
```

```
Epoch 491/500
1/1             0s 25ms/step - loss:
1.9921e-05
Epoch 492/500
1/1             0s 25ms/step - loss:
2.0056e-05
Epoch 493/500
1/1             0s 25ms/step - loss:
1.9907e-05
Epoch 494/500
1/1             0s 24ms/step - loss:
1.9428e-05
Epoch 495/500
1/1             0s 24ms/step - loss:
1.8758e-05
Epoch 496/500
1/1             0s 25ms/step - loss:
1.8160e-05
Epoch 497/500
1/1             0s 25ms/step - loss:
1.7783e-05
Epoch 498/500
1/1             0s 44ms/step - loss:
1.7596e-05
Epoch 499/500
1/1             0s 28ms/step - loss:
1.7484e-05
Epoch 500/500
1/1             0s 26ms/step - loss:
1.7348e-05
```

[11]:
```
<keras.src.callbacks.history.History at 0x1417cb620>
```

[12]:
```
best_ber = float('inf')

for iteration in range(10):
    print(f"====== Iteration {iteration} ======")

    # symbDPD = symbTx_nn if iteration == 0 else dpd_model.predict(symbTx_nn,␣
 ↪verbose=0)
    symbDPD = dpd_model.predict(symbTx_nn, verbose=0)


    y_CPR_1, d= simulate_optical_system(merge_i_q(symbDPD).flatten(),␣
 ↪paramSymb, paramPulse,paramIQM,sigLO, paramCh, paramLO, paramFE,␣
 ↪paramPD,paramRxPulse,paramDec
```

```
                              ,paramEDC,paramEq,paramCPR, Data_Aided=True,␣
↪PA_enable=True)


   ber = perf_calc(symbTx, y_CPR_1, d, M, paramSymb, paramDec)

   if ber < best_ber:
       dpd_model.save_weights("best_model.weights.h5")
       best_ber = ber


   y_CPR_1_nn = split_i_q(y_CPR_1).reshape(-1, seq_length, 2)


   # y = (y_CPR_1 - np.mean(y_CPR_1)) / np.std(y_CPR_1)
   # y_nn = split_i_q(y).reshape(-1, seq_length, 2)

   dpd_model.fit(y_CPR_1_nn, symbDPD, epochs=100, verbose=0) # ILA


   ## PLEASE IGNORE THE BELOW CODE
   # aux_model.fit(symbDPD, y_CPR_1_nn, epochs=200, verbose=0)
   # aux_model.trainable = False
   # dla_cascade.compile(optimizer=tf.keras.optimizers.
↪Adam(learning_rate=1e-3), loss='mse')

   # dla_cascade.fit(symbTx_nn, symbTx_nn, epochs=100, verbose=0)
   # aux_model.trainable = True
   # dla_cascade.compile(optimizer=tf.keras.optimizers.
↪Adam(learning_rate=1e-3), loss='mse')
```

```
====== Iteration 0 ======

  0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.041287.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.035405.
rde - training stage #1
rde MSE = 0.023937.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
```

```
Estimated linewidth: 370.594 kHz

 SER: 7.222e-04,
 BER: 1.806e-04
 SNR: 19.495 dB
 EVM: 1.128 %
 Qfactor: 5.523,
====== Iteration 1 ======

  0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.049518.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.043334.
rde - training stage #1
rde MSE = 0.014186.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 255.114 kHz

 SER: 2.222e-04,
 BER: 5.556e-05
 SNR: 22.278 dB
 EVM: 0.599 %
 Qfactor: 5.871,
====== Iteration 2 ======

  0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.050534.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.044381.
rde - training stage #1
rde MSE = 0.013378.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 253.335 kHz

 SER: 2.000e-04,
```

```
 BER: 5.000e-05
 SNR: 22.489 dB
 EVM: 0.567 %
 Qfactor: 5.900,
====== Iteration 3 ======

  0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.050391.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.044259.
rde - training stage #1
rde MSE = 0.013288.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 251.371 kHz

 SER: 2.111e-04,
 BER: 5.278e-05
 SNR: 22.533 dB
 EVM: 0.560 %
 Qfactor: 5.885,
====== Iteration 4 ======

  0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.050484.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.044362.
rde - training stage #1
rde MSE = 0.013235.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 249.531 kHz

 SER: 2.000e-04,
 BER: 5.000e-05
 SNR: 22.554 dB
 EVM: 0.557 %
```

```
 Qfactor: 5.900,
====== Iteration 5 ======

  0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.050635.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.044516.
rde - training stage #1
rde MSE = 0.013183.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 244.376 kHz

 SER: 2.111e-04,
 BER: 5.278e-05
 SNR: 22.571 dB
 EVM: 0.554 %
 Qfactor: 5.885,
====== Iteration 6 ======

  0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.050764.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.044649.
rde - training stage #1
rde MSE = 0.013238.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 234.559 kHz

 SER: 2.111e-04,
 BER: 5.278e-05
 SNR: 22.567 dB
 EVM: 0.554 %
 Qfactor: 5.885,
====== Iteration 7 ======
```

```
  0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.050894.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.044781.
rde - training stage #1
rde MSE = 0.013188.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 242.536 kHz

 SER: 2.000e-04,
 BER: 5.000e-05
 SNR: 22.585 dB
 EVM: 0.552 %
 Qfactor: 5.900,
====== Iteration 8 ======

  0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.050999.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.044884.
rde - training stage #1
rde MSE = 0.013084.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 249.653 kHz

 SER: 1.889e-04,
 BER: 4.722e-05
 SNR: 22.600 dB
 EVM: 0.550 %
 Qfactor: 5.916,
====== Iteration 9 ======

  0%|          | 0/1 [00:00<?, ?it/s]

Running CD compensation…
```

```
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
da-rde - training stage #0
da-rde pre-convergence training iteration #0
da-rde MSE = 0.051049.
da-rde pre-convergence training iteration #1
da-rde MSE = 0.044938.
rde - training stage #1
rde MSE = 0.013196.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 238.854 kHz

 SER: 2.111e-04,
 BER: 5.278e-05
 SNR: 22.587 dB
 EVM: 0.551 %
 Qfactor: 5.885,
```

[13]:
```python
def optical_chain_w_dpd(symbTx, DPD_ON=False, Data_Aided=False, PA_enable=True):
    symbTx_nn = split_i_q(symbTx)
    symbTx_nn = symbTx_nn[:reshabe_len*seq_length].reshape(-1, seq_length, 2)
    symbDPD = dpd_model.predict(symbTx_nn, verbose=0) if DPD_ON==True else␣
 ↪symbTx_nn
    y_CPR_1, d= simulate_optical_system(merge_i_q(symbDPD).flatten(),␣
 ↪paramSymb, paramPulse,paramIQM,sigLO, paramCh, paramLO, paramFE,␣
 ↪paramPD,paramRxPulse,paramDec
                                    ,paramEDC,paramEq,paramCPR,␣
 ↪Data_Aided=Data_Aided, PA_enable=PA_enable)
    return y_CPR_1,d
```

0.2 Performance Evaluation

[ ]:
```python
def test(new_seed):
    dpd_model.load_weights("best_model.weights.h5")
    paramSymb.seed = new_seed
    symbTx = symbolSource(paramSymb)
    y_CPR_1_dpd, d_dpd = optical_chain_w_dpd(symbTx, DPD_ON=True,␣
 ↪Data_Aided=False, PA_enable=True)
    y_CPR_1, d = optical_chain_w_dpd(symbTx, DPD_ON=False, Data_Aided=False,␣
 ↪PA_enable=True)

    print("====== Perf Without DPD (Data-Blind RX) ======")
    perf_calc(symbTx, y_CPR_1, d, M, paramSymb, paramDec)


    print("====== Perf With DPD (Data-Blind RX): ======")
```

```
    perf_calc(symbTx, y_CPR_1_dpd, d_dpd, M, paramSymb, paramDec)


test(444) #  BER: 2.139e-03 - sin activation, PA=3, mzmscale=0.66, datablind ///
 ↪ it's 2.564e-03 with RELU
```

```
  0%|          | 0/1 [00:00<?, ?it/s]
Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
cma - training stage #0
cma pre-convergence training iteration #0
cma MSE = 0.461529.
cma pre-convergence training iteration #1
cma MSE = 0.454642.
rde - training stage #1
rde MSE = 0.013582.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 255.605 kHz
  0%|          | 0/1 [00:00<?, ?it/s]
Running CD compensation…
CD filter length: 46 taps, FFT size: 64
Running adaptive equalizer…
cma - training stage #0
cma pre-convergence training iteration #0
cma MSE = 0.431907.
cma pre-convergence training iteration #1
cma MSE = 0.425488.
rde - training stage #1
rde MSE = 0.024351.
Running frequency offset compensation…
Estimated frequency offset (MHz): [128.24]
Running BPS carrier phase recovery…
Estimated linewidth: 396.243 kHz
====== Perf Without DPD (Data-Blind RX) ======
 SER: 1.646e-01,
 BER: 7.750e-02
 SNR: 5.526 dB
 EVM: 26.055 %
 Qfactor: 1.529,
====== Perf With DPD (Data-Blind RX): ======
 SER: 1.007e-02,
 BER: 2.564e-03
 SNR: 19.392 dB
```

```
EVM: 1.148 %
Qfactor: 4.470,
```